



MASTER'S DEGREE THESIS

Dipartimento di Informatica
Corso di Laurea Magistrale in Informatica

RON something..

Author:

Vincenzo Gargano

Supervisors:

Prof. Claudio Gallicchio

Dott. Andrea Ceni

Dott. Andrea Cossu,

Academic Year 2024/2025

Abstract

...

Contents

1	Introduction	7
1.1	Motivation and Objectives	7
1.2	Reservoir Computing	7
1.3	Scientific contribution	8
1.4	Structure of the work	8
2	Background	9
2.1	The Inherent Dynamism of Natural Systems	9
2.2	Dynamical Systems and Chaos Theory	10
2.2.1	Discretization	11
2.3	Overview of Machine Learning	11
2.3.1	Learning Paradigms	14
2.3.2	Overfitting and Generalization	14
2.3.3	Optimization	14
2.4	Structured Probabilistic Models (Graphical Models)	14
2.5	Deep Recurrent Neural Networks	15
2.6	Deep Reservoir Computing	15
3	Problem Statement and Related works	17
4	Stacking RON Layers	19
5	Implementation and Experimental results	21
6	Conclusion and Future Works	23

Chapter 1

Introduction

1.1 Motivation and Objectives

Nowadays, Artificial Intelligence (AI) became the main driver of innovation in many fields, at the point that the growth of research became exponential in the last decade. This fully aligns with the current state of our world, in fact, we are observing a paradigm shift in the way we interact with technology (or world), through the use of AI.

Within AI, whose main goal is not only to understand *intelligence*, but to embed it into agents capable of performing complex tasks through *rational reasoning*, *planning* and *acting*, the subfield of Machine Learning (ML) has emerged as the dominant approach to mimicking biological intelligent behaviour. Neural Networks (NN) *learn* complex patterns from large amounts of real-world data, instead of relying on explicit rule based approaches, in order to perform *inference* and return prediction on unseen data.

The last years have seen a surge in popularity of generative models, after a first wave of discriminative models that made popular the field of Deep Learning (DL), and introduced GPU computation in 2009 [1]. Particularly, deep generative models have been applied with unbelievable results in many fields, from image generation with *Diffusion Models* [2] to text generation with *Transformers* [3] and protein folding with *AlphaFold* [4] and many others domains, achieving state of the art results. However, deep learning models are known to be expensive in both training and inference, requiring large amount of computational resources (GPU production) and energy consumption [5].

Forse dovrei concentrarmi più su RNN dall'inizio?

1.2 Reservoir Computing

Reservoir Computing (RC) is a computational framework introduced in the early 2000s with Echo State Networks (ESN) [6] and Liquid State Machines (LSM) [7], these models aim to simplify the training of RNNs and at the same time exploit inherent dynamical properties of real world physical system in order to obtain a "reservoir" of rich dynamics that can be used to solve complex task, at the same time

Forse in questa sezione devo essere ancora più generico e parlare solo dei vantaggi e le motivazioni d'uso per spiegare tutto meglio nel capitolo di background

we reduce the training cost by only performing a linear regression on the output layer. The main idea is that usually we need to train the recurrent connection in the network, but RC avoid this by using fixed high dimensional dynamical system, that project the input signal into an high dimensional space, where a simple approach as pseudoinverse or ridge regression is sufficient to learn the linearized representation of the input signal.

With RC we can solve chaotic time series prediction, speech recognition, robot control with ease with minimal training cost, and we can even use them as a generative model with techniques like Teacher Forcing [8].

1.3 Scientific contribution

1.4 Structure of the work

elenco puntato con quello che si dirà in ogni capitolo della tesi

Chapter 2

Background

In this chapter we will review the main concepts and theory needed to understand the rest of this thesis. We will start with a brief introduction of the Dynamical Systems (DS) theory and a brief overview of Chaos theory and Machine Learning concepts applied to those type of systems. Then we will move to a more specific topic, about RC and the ESN model, with some emphasis on the Deep counterparts of both models treated in this work.

2.1 The Inherent Dynamism of Natural Systems

Our universe is essentially a complex non-linear dynamical system, and since Newton in mid 1600s, used differential equations as a method to solve the two body problems, but many natural phenomena are essentially Complex Systems, from small components of the environment there are interdependences (CS) which exhibit dynamics that evolve through time: such as motion of planets, flow of fluids, weather changes, growth of population and many others phenomena.

Dynamics is a fundamental piece of reality and modeling time dependent system evolution, is crucial to understand the world around us: eg. the design of control system, stock market and also the human brain can be modeled as a dynamical system. The methodological framework developed through the preceding century have been specifically designed to analyze sequential data structures, commonly referred as *time series*, These approaches encompass techniques such as Autoregressive (AR) models (i should cite but cannot find source) which employ linear regression on data sequences, extending to more sophisticated frameworks including State-Space Models [9], originally developed for Control Theory applications, and Hidden Markov Models (HMM) [10]. The latter served as foundational precursors to Recurrent Neural Networks [11] (RNNs). The aforementioned models, are the basis of modern neural based sequential modeling.

questo va nel background

devo approfondire i sistemi complessi?

forse questa parte s framework non va messa qui

2.2 Dynamical Systems and Chaos Theory

A dynamical system is a system which evolves over time, we can describe it's changes through **state variables**, usually many models are constructed such that only the fundamental state variables are considered, for example *biological spiking neurons* modeled as in [12] are described by a simple 2-D dimensional ODE, where the two state variables are the membrane potential and a recovery variable. ignoring existing other possible variables such gradient of ion concentration, temperature; for the sake of obtaining a large scale simulation.

There are two types of dynamical systems: *Differential Equations*, which are continuous in time, and *Difference Equations* or *Iterated Maps* which are discrete in time. The most used in a wide range of sciences fields are the former, which are further distinguished in Ordinary Differential Equations (ODE) and Partial Differential Equations (PDE), the difference is that in PDE we have two or more independent variables.

This thesis will focus on ODEs since our models fell into that category, but we will also use *discretization methods* (link to the section maybe) to obtain it's approximated solution.

Formally we define an ODE as:

$$\frac{d\mathbf{x}}{dt} = f(\mathbf{x}, t) \quad (2.1)$$

where $\mathbf{x} \in \mathbb{R}^n$ is the state vector, t is the time and $f : \mathbb{R}^n \times \mathbb{R} \rightarrow \mathbb{R}^n$ is a vector field grouping all the state variables that describe the evolution of the system.

questo va tolto, forse meglio non andare troppo in dettaglio in queste cose

An example of ODE is the well known Dampened Harmonic Oscillator:

$$\frac{d^2x}{dt^2} + 2\beta \frac{dx}{dt} + \omega_0^2 x = 0 \quad (2.2)$$

anche questo va tolto?

where x is the position of the oscillator, β is the damping coefficient and ω_0 is the natural frequency of the oscillator.

A Differential Equation is *autonomous* if the function f does not depend explicitly on time, otherwise is *non-autonomous*. We can switch from a non-autonomous to an autonomous system by adding time as a new variable to the state vector, for example the non-autonomous system of equation 2.2 can be rewritten with a change of variable.

Modeling

Modeling is a central methodology in the study of complex systems, translating real world observations and hypotheses into a formalized mathematical model, even though simplified, can be crucial for studying emergent behaviours, particularly in natural systems where we cannot change the initial state, or the environment variables that influence it.

The Hodgkin-Huxley model [13] describes the dynamics of neuronal membrane potential through an equivalent electrical circuit. The membrane can be represented as a capacitor C_m in parallel with several ion channels, each modeled as a voltage-dependent conductance in series with a battery representing the ion's reversal potential.

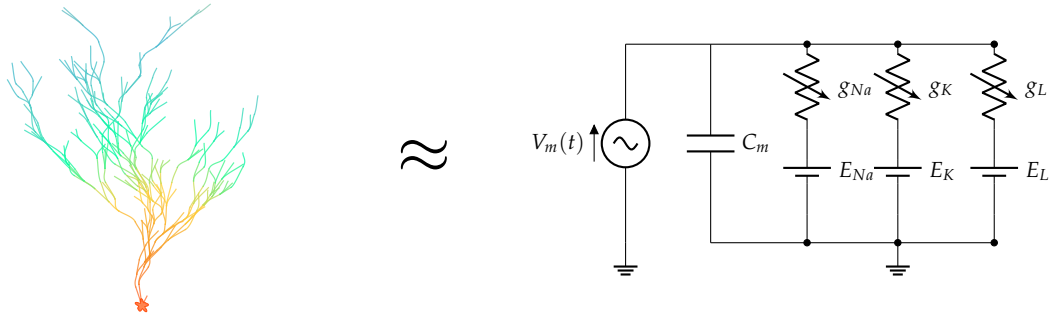


FIGURE 2.1: Biological neuron and its Hodgkin-Huxley equivalent circuit representation.

The corresponding differential equation describing the membrane potential dynamics is:

$$C_m \frac{dV_m}{dt} = I_{ext} - g_{Na}(V_m - E_{Na}) - g_K(V_m - E_K) - g_L(V_m - E_L) \quad (2.3)$$

where I_{ext} is the external input current, and each conductance g_i is voltage and time dependent, governed by additional gating variables.

2.2.1 Discretization

2.3 Overview of Machine Learning

Machine Learning (ML) is a subfield of Artificial Intelligence (AI) that focuses on statistical methods and algorithms capable to *learn* from past experiences. First we need a rigorous definition of what means to learn; Mithell [14] define ML as "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E ".

ML can tackle a much wider range of tasks that are difficult to solve with explicit rules based programming, in a certain sense it bypasses some level of abstraction, we're not learning the task T directly, but we're sampling from the E space, **examples** which are collection of **features** that are qualitative or quantitative properties of the data.

A formal definition of a machine learning model is given below:

Definition 1. Let $P(\mathbf{x})$ be an independent identically distributed (i.i.d.) unknown probability distribution over the input space $\mathcal{X}$. A dataset is a finite sample drawn from this distribution, $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^N \subset \mathcal{X}$, where $\mathbf{x}_i \in \mathbb{R}^d$ are data points (or input features).

A **machine learning model** is a function $f : \mathbb{R}^d \rightarrow \mathcal{O}$ parameterized by Θ , which maps inputs to an output space $\mathcal{O}$. The nature of the model and its goal is defined by the learning paradigm:

- In the **supervised** setting, the data is drawn from a joint distribution $P(\mathbf{x}, y)$, so $\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, with targets $y_i \in \mathcal{Y}$. Here, $\mathcal{O} \equiv \mathcal{Y}$ (e.g., $\mathbb{R}$ for regression or $\{0, 1\}$ for classification), and the goal is to find parameters Θ^* that minimize a loss function $\mathcal{L}(y, f(\mathbf{x}; \Theta))$ quantifying the discrepancy between predictions and true labels.
- In the **unsupervised** setting, the data consists only of inputs $\mathbf{x}_i$. The output space $\mathcal{O}$ is defined by the specific task (e.g., $\mathbb{R}^k$ for dimensionality reduction, a set of clusters for clustering, or $\mathbb{R}^d$ for generative models). The goal is to find parameters Θ^* that optimize an objective function $\mathcal{J}(f(\mathbf{x}; \Theta), \mathcal{D})$, which captures desired structural properties of the data, such as reconstruction error, cluster cohesion, or data likelihood.

In both cases, the ultimate objective is for the model $f(\mathbf{x}; \Theta^*)$ to generalize well to new, unseen data $\mathcal{D}_{\text{test}} \sim P(\mathbf{x})$.

As we've just explained, models can learn the data distribution in three main different styles:

1. **Supervised Learning**, where we aim for accurate predictions.
2. **Unsupervised Learning**, tries to find a compact description of the data
3. **Reinforcement Learning**, where an agent recursively interacts with an environment in order to maximize a cumulative reward signal, through a certain policy.

c'è ne sono molti altri: semi-supervised, active learning etc., nomino?

Understanding learning is crucial to choose the right modeling technique and architecture for a given task. In order to have a more compact picture of learning let's decompose the more general learning objective: $P(\mathbf{X}, Y)$

Learning, Generalizing and Overfitting

The joint distribution $P(\mathbf{X}, Y)$ decomposes as:

$$P(\mathbf{X}, Y) = P(Y|\mathbf{X})P(\mathbf{X}) = P(\mathbf{X}|Y)P(Y) \quad (2.4)$$

Vorrei evitare di spingere bayes, ma come introduco questi concetti?

$$\text{Joint} = (\text{Posterior}) \cdot (\text{Marginal}) = (\text{Likelihood}) \cdot (\text{Prior})$$

In machine learning, what is most relevant is often the *posterior distribution*, telling us how much our hypothesis y is likely after we've seen the data x . But we could also be interested to approximate the unknown distribution itself, to generate new samples from it, in that case we're interested in the *likelihood* and the *prior* distributions. This is the main catch between learning differences of *Generative* and *Discriminative* models.

By rewriting in terms of posterior distribution we get what is commonly referred as the **Bayes' Theorem**.

citabayes

Bayesian Decision Theory [15] provides a statistical framework for the design of many machine learning algorithms. The way model parameters are estimated often reflects whether the approach is *Generative* or *Discriminative*.

¹ Discriminative models rely often on conditional distribution, the posterior, and whenever we can incorporate prior knowledge on the parameters, inference is performed through **Maximum A Posteriori** (MAP) estimation 2.5, which seeks the best hypothesis (θ) given the data and prior knowledge.

¹However, this correspondence is not absolute [15, p. 312, Ch. 13]: logistic regression can also be trained by pure MLE, and generative models can incorporate priors and thus be estimated via MAP.

$$\theta^* = \arg \max_{\theta} P(\theta | \mathcal{D}) = \arg \max_{\theta} P(\mathcal{D} | \theta) P(\theta) \quad (2.5)$$

On the other hand generative models learn the joint distribution, and are typically trained by a **Maximum Likelihood Estimation** (MLE) 2.6, fitting the parameters that maximize the likelihood of the data.

$$\theta^* = \arg \max_{\theta} P(\mathcal{D} | \theta) \quad (2.6)$$

We've focused on how to learn the parameters of a model, however the methods of learning we've seen so far, try to find an exact solution with respect to the training data, this does not guarantee **generalization**, however there are ways even for bayesian models to achieve generalization.

Since we use the loss function as a proxy for performance, and this function can be a similarity or distance measure it introduces some error, we can

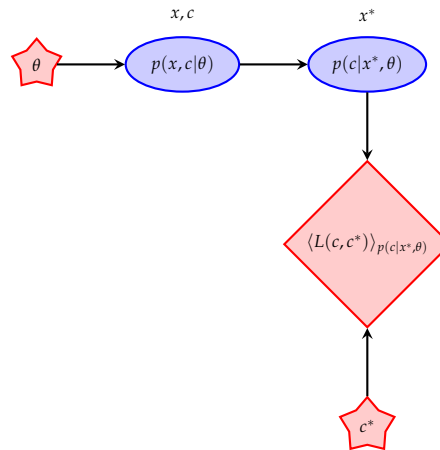


FIGURE 2.2: Probabilistic graphical model representation

2.3.1 Learning Paradigms

2.3.2 Overfitting and Generalization

2.3.3 Optimization

2.4 Structured Probabilistic Models (Graphical Models)

Structured Probabilistic Models are used to describe a probability distribution using a graph (from graph theory) structure, which nodes represent random variables and edges represent dependencies between them.

What need to add: Directed and Undirected, inference and learning basics, few famous models (HMM, RBM, CRF Energy-Based Models (EBM), why structured models are so useful : Information flow, interpretability, prior knowledge, etc.)

2.5 Deep Recurrent Neural Networks

Deep Learning had a great impact on the field of machine learning and artificial intelligence. When we consider a deep learning model, we are referring to a model that has multiple layers of neurons. The most common type of deep learning model is the Multilayer Perceptron (MLP), which consists of multiple layers of fully connected neurons. By adding layers to these models we are able to learn representation at different scales, each layer take the pattern from the previous layers and is capable to learn a more complex pattern. In this work, we will focus on a set of deep learning models known as Recurrent Neural Networks (RNNs). [11] RNNs are a type of neural network that is designed to handle sequential data. Depth in these architectures usually refers to the number of timestep we unroll the network through time (or sequence). But we can add depth in more than one way, one of those is to stack multiple layers of RNNs on top of each other sequentially.

citations needed

Change this image to make a new one or cite it

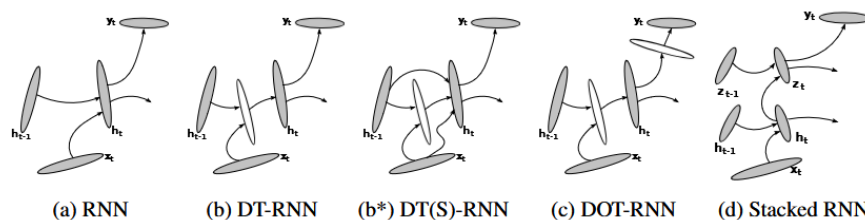


Figure 2: Illustrations of four different recurrent neural networks (RNN). (a) A conventional RNN. (b) Deep Transition (DT) RNN. (b*) DT-RNN with shortcut connections (c) Deep Transition, Deep Output (DOT) RNN. (d) Stacked RNN

FIGURE 2.3: Depths in Neural Network

2.6 Deep Reservoir Computing

Reservoir Computing (RC) is a framework introduced by , that is designed to simplify the training of Recurrent Neural Networks. The main point is that we do not need to train the recurrent connections of the network, but only an output layer called the readout.

cite

Chapter 3

Problem Statement and Related works

Chapter 4

Stacking RON Layers

Chapter 5

Implementation and Experimental results

Chapter 6

Conclusion and Future Works

Bibliography

- [1] Rajat Raina, Anand Madhavan, and Andrew Y. Ng. "Large-scale deep unsupervised learning using graphics processors". In: *Proceedings of the 26th Annual International Conference on Machine Learning*. ICML '09. Montreal, Quebec, Canada: Association for Computing Machinery, 2009, pp. 873–880. ISBN: 9781605585161. DOI: [10.1145/1553374.1553486](https://doi.org/10.1145/1553374.1553486). URL: <https://doi.org/10.1145/1553374.1553486>.
- [2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: [2006.11239](https://arxiv.org/abs/2006.11239) [cs.LG]. URL: <https://arxiv.org/abs/2006.11239>.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need*. 2023. arXiv: [1706.03762](https://arxiv.org/abs/1706.03762) [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- [4] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, et al. "Highly accurate protein structure prediction with AlphaFold". In: *nature* 596.7873 (2021), pp. 583–589.
- [5] Razvan Pascanu, Caglar Gulcehre, Kyunghyun Cho, and Y. Bengio. "How to Construct Deep Recurrent Neural Networks". In: (Dec. 2013).
- [6] Herbert Jaeger. "The "echo state" approach to analysing and training recurrent neural networks-with an erratum note". In: *Bonn, Germany: German national research center for information technology gmd technical report* 148.34 (2001), p. 13.
- [7] Chrisantha Fernando and Sampsa Sojakka. "Pattern recognition in a bucket". In: *European conference on artificial life*. Springer. 2003, pp. 588–597.
- [8] Ronald J. Williams and David Zipser. "A Learning Algorithm for Continually Running Fully Recurrent Neural Networks". In: *Neural Computation* 1.2 (1989), pp. 270–280. DOI: [10.1162/neco.1989.1.2.270](https://doi.org/10.1162/neco.1989.1.2.270).
- [9] Rudolph Emil Kalman. "A new approach to linear filtering and prediction problems". In: (1960).
- [10] Leonard E Baum and Ted Petrie. "Statistical inference for probabilistic functions of finite state Markov chains". In: *The annals of mathematical statistics* 37.6 (1966), pp. 1554–1563.
- [11] Jeffrey L. Elman. "Finding structure in time". In: *Cognitive Science* 14.2 (1990), pp. 179–211. ISSN: 0364-0213. DOI: [https://doi.org/10.1016/0364-0213\(90](https://doi.org/10.1016/0364-0213(90)

- 90002-E. URL: <https://www.sciencedirect.com/science/article/pii/S036402139090002E>.
- [12] E.M. Izhikevich. "Simple model of spiking neurons". In: *IEEE Transactions on Neural Networks* 14.6 (2003), pp. 1569–1572. DOI: [10.1109/TNN.2003.820440](https://doi.org/10.1109/TNN.2003.820440).
- [13] Alan L Hodgkin and Andrew F Huxley. "A quantitative description of membrane current and its application to conduction and excitation in nerve". In: *The Journal of physiology* 117.4 (1952), p. 500.
- [14] Tom M Mitchell. *Machine learning*. Vol. 1. 9. McGraw-hill New York, 1997.
- [15] D. Barber. *Bayesian Reasoning and Machine Learning*. 04-2011. In press. Cambridge University Press, 2011. URL: <http://www.cs.ucl.ac.uk/staff/d.barber/brml>.